HackMatrix Final Round Participant Guide

Two additions for each problem statement

The final 24-hour round extends the working prototypes prepared before the event. Complete both additions for your selected problem statement and integrate them into that prototype. Addition 1 deepens the system behavior; Addition 2 introduces a specific AI or ML task that supports it.

How to use this guide

- Choose the entry matching your existing PS code. Teams complete their selected PS, not all sixteen entries. The index links directly to each requirement page.
- Read the common requirements, both additions, their connection, the scope and the demonstration conditions. The original PS outcomes remain required.
- Use the listed scope as a minimum working demonstration. A bounded implementation is acceptable when its constraints, model behavior and evaluation are complete.

What the final round changes

Prepared baseline work may be reused. Record the repository commit and existing capabilities at the start of the final round, then identify the new implementation, adaptation, integration and evaluation in the final submission. Disclose any addition already implemented before the round.

For this round, this preparation policy replaces the original booklet sentence requiring the entire solution to be built within the 24-hour window. The two additions and attributable final-round work are the focus of this addendum.

Data and model choices

Organizers will not supply datasets or test scenarios. Teams must source their inputs and build their own scenario suite. Public data, existing permitted data, documented synthetic records and simulated events are allowed.

Choose a model appropriate to your task and available resources. Pretrained models and APIs are allowed within this brief; disclose their identity and role. Training a large model from scratch is not required. Where a PS explicitly requires training or adaptation, show that step and its resulting artifact.

Common implementation and submission requirements

Demonstrate the two additions together

- The AI or ML output must affect the new workflow, ranking, plan, investigation or measurement. Show its input, output and the resulting application behavior.
- Keep exact policy rules, access permissions, physical limits and resource constraints enforced. Learned predictions support decisions; they do not replace these controls.
- Cover every condition listed for your PS, using at least four distinct end-to-end scenarios. Conditions may share a scenario if each outcome is visible. Include normal operation, changing information and failure or uncertainty.

Evaluate the model honestly

- Use a simpler baseline on the same held-out inputs. Keep model fitting, prompt selection, threshold tuning and calibration separate from final evaluation. Four demo scenarios alone are not a model benchmark.
- For forecasts, train and tune on earlier periods and evaluate on later periods. Group related documents, connected cases, users or original images when relevant; derived copies must not leak across splits.
- Declare labels, sample counts, class balance, units, thresholds and denominators. Check reference labels independently; a model must not certify its own answers. Synthetic generators must hide outcomes and future events from predictors.
- Report uncertainty with evidence. Describe failure cases and whether inputs are real, replayed, transformed or simulated. Results on synthetic inputs demonstrate behavior under those assumptions, not real-world deployment accuracy.

Submit a reproducible evidence package

- Repository start and final commits, a concise change summary, setup instructions and the commands needed to run the application and evaluator.
- Data sources and permissions where relevant, schemas, reference labels, synthetic generation assumptions and random seeds, plus the evaluation split or grouped identifiers.
- Model or API identity and version, relevant parameters or prompts, training/adaptation details where applicable, and saved predictions with expected outcomes.
- A baseline comparison, the PS-specific output artifacts, scenario replay logs and at least one documented limitation or unsuccessful case.

Evaluation emphasizes correctness, a justified model contribution, integration and credible evidence. No universal accuracy threshold or preferred architecture applies across the sixteen PSs.

ENERGY | ENR-02

ENR 02 Renewable Distribution Grid Digital Twin

Preserve the electrical network, solar/load inputs and physical power-flow checks. Add control over time and a learned approximation used to screen candidate actions.

Addition 1 Control across multiple time steps

- Plan over at least eight time steps while tracking battery state of charge, power limits, network limits and permitted switching or curtailment actions.
- Replan as observed solar generation and load differ from forecasts. Show state transitions and before/after power-flow results for the action sequence.
- Identify infeasible periods and any fallback explicitly. Validate every selected action with the physical solver before presenting it as safe.

Addition 2 Learning a power flow surrogate

- Generate labeled operating states with the existing solver and train a surrogate to estimate node voltages and line loading or screen risky states.
- Use it in candidate evaluation while preserving final solver verification. Declare its supported operating range and fallback for unsupported inputs.
- Evaluate on separately generated unseen operating conditions, including states near equipment limits. Report prediction error, unsafe states mistaken for safe, and execution time.

Scope and data: Use the existing simplified feeder and declared equipment limits. Solver-generated data are allowed; use distinct development and test operating ranges or parameter combinations.

Required demonstration and evidence

Demonstrate forecast error, battery depletion or saturation, a near-limit network state, and an infeasible period. Show surrogate disagreement and the solver response.

Submit solver configurations, generated-state splits, surrogate artifacts, state-of-charge traces, action logs, error/latency results and final verification outputs.